QuVaultX-PC: A Programmable Cryptography Chiplet Architecture

From Lattices to Cryptomata — Silicon Foundations for Zero-Knowledge, Homomorphic Computation, and Obfuscation

Kiprop Yego
Independent Research | Chiplet Security Systems | QuVaultX
kiprop.yego@quvaultx.dev

Conceptual Architecture White Paper — Draft v0.2 — March 2026

Abstract

We present QuVaultX-PC (*Programmable Cryptography* edition), an extended chiplet architecture that elevates the post-quantum cryptographic foundations of the original QuVaultX chiplet [1] into a fully programmable cryptographic substrate. Drawing on the *programmable cryptography* framework articulated by 0xPARC [2], we propose a five-pillar silicon architecture that accelerates not merely the NIST post-quantum standard suite, but the full hierarchy of modern cryptographic machinery: **(i)** a reconfigurable Zero-Knowledge Proof Engine (ZKPE) targeting PLONK/Groth16/Nova-class proof systems at 10× native computation overhead; **(ii)** an extended Fully Homomorphic Encryption Substrate (FHES) with verifiable-FHE gadgets and ORAM-backed secure memory; **(iii)** a Multi-Party Computation Fabric (MPCF) supporting garbled-circuit and threshold-FHE protocols with intra-chiplet party isolation; **(iv)** a Functional Encryption and Obfuscation Engine (FEOE) providing hardware-accelerated functional decryption, k-linear-map arithmetic, and a sandboxed indistinguishability-obfuscation (i$\mathcal{O}$) execution environment; and **(v)** the Cryptomaton Execution Unit (CEU), a novel proposed microarchitecture for running non-interactive, privacy-preserving autonomous programs directly on silicon. The chiplet integrates over a UCIe 3.0 die-to-die fabric at 64 GT/s, targeting 112 mm^2 in TSMC N5 FinFET alongside the Ogopogo quad-chiplet RISC-V manycore. We introduce the concept of a *cryptomaton-on-silicon*: a hardware execution context in which a program can accept public input, maintain private state opaque even to the hosting die, and emit provably correct, selectively decryptable output—realising in silicon the "beyond primitive computing" vision of programmable cryptography.

Index Terms— Programmable cryptography, chiplet, zero-knowledge proofs, zkSNARK, PLONK, FHE, MPC, functional encryption, indistinguishability obfuscation, cryptomaton, NTT accelerator, UCIe, post-quantum, RISC-V, 2.5D integration.

1 Introduction

In August 2024, NIST finalised three post-quantum cryptographic standards [6], completing an eight-year standardisation effort. QuVaultX [1] was conceived as a first-class chiplet to accelerate this standard suite inline with a host manycore. Yet the cryptographic horizon extends far beyond key encapsulation and digital signatures.

The 0xPARC research collective [2] articulates a vision they term *programmable cryptography*: a future in which cryptography is not a fixed menu of primitives (AES, RSA, Kyber) but a programmable substrate capable of expressing arbitrary trust relationships. They identify three ascending layers of capability:

1. **Zero-Knowledge Proofs and the Universal Protocol.** Arbitrary statements proved without revealing witnesses; cost today is $\approx 10^6\times$ the native computation, target 10×.
2. **MPC, FHE, and the Hallucinated Server.** Distributed virtual servers whose internal state no participant can read.
3. **Obfuscation, Functional Encryption, and**

Cryptomata. Autonomous, non-interactive cryptographic agents whose private state is opaque even to the machine that executes them.

The first two layers are theoretically sound and practically improving; the third is theoretically possible (via indistinguishability obfuscation $i\mathcal{O}$ and functional encryption FE) but not yet practical.

Our contribution. We ask: *what silicon substrate would accelerate all three layers simultaneously, and what microarchitectural primitives would make the third layer tractable?* QuVaultX-PC answers this question. It extends the original QuVaultX post-quantum chiplet with four additional subsystems targeting the full programmable-cryptography stack, culminating in the *Cryptomaton Execution Unit*—a novel hardware abstraction for privacy-preserving autonomous program execution.

The remainder of this paper is organised as follows. §2 motivates the extended threat and capability model. §3 presents the top-level architecture. §4–§8 detail each of the five pillars. §9 covers UCIe integration. §10 presents implementation estimates. §11 compares with the state of the art. §12 outlines the open-source roadmap. §13 concludes.

2 Capability and Threat Model

2.1 Beyond the Quantum Adversary

The original QuVaultX threat model centred on the *cryptographically relevant quantum computer* (CRQC) [1]. QuVaultX-PC retains this model and extends it with three additional adversary classes that programmable cryptography is designed to defeat.

2.1.1 The Honest-but-Curious Server

In cloud-hosted AI inference, genomic analysis, and financial modelling, the *service provider* should learn nothing about the client's data. Defeating this adversary requires FHE or MPC.

2.1.2 The Malicious Co-Party

In multi-party protocols (joint auctions, private set intersection, federated learning), any one participant may deviate from the protocol. Defeating this adversary requires *verifiable* MPC and FHE with succinct proof of correct evaluation.

2.1.3 The Compromised Host

A cryptomaton's execution host may be malicious or compromised. The cryptomaton's internal state must remain opaque even to the die on which it runs. This adversary is defeated only by a combination of $i\mathcal{O}$, FE, and hardware isolation.

2.2 The Programmable Cryptography Capability Stack

Definition 2.1 (Programmable Cryptographic Primitive)**.** A *programmable cryptographic primitive* is a cryptographic scheme $\Pi = (\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Eval}, \mathsf{Dec})$ in which Eval accepts an arbitrary polynomial-time computable function f as a parameter, such that $\mathsf{Dec}(\mathsf{sk}_f, \mathsf{Enc}(\mathsf{pk}, x)) = f(x)$ for an appropriate function-specific key sk_f.

This definition subsumes ZK proofs (where f is a circuit satisfiability check), FHE (where f is arbitrary and $\mathsf{sk}_f = \mathsf{sk}$), and FE (where f is arbitrary and sk_f is function-specific).

2.3 Performance Targets

Table 1 states the performance targets for each layer, derived from 0xPARC's stated goals [2] translated into silicon requirements.

Table 1: Programmable Cryptography Performance Targets

Layer	Operation	CPU	Target
ZK	PLONK prove (1M gates)	∼60s	<200ms
ZK	Groth16 prove	∼30s	<100ms
ZK	Proof verify	∼10ms	<0.1ms
FHE	CKKS Bootstrap (2^{16})	>200s	∼1s
FHE	Verifiable HomMul	N/A	<5ms
MPC	Garbled circuit (1M gates)	∼10s	<50ms
FE/iO	FE.Dec (circuit depth 20)	N/A	<10ms
Cryptomaton	Step (100 gates)	N/A	<1ms

3 Top-Level Architecture

3.1 Design Philosophy

QuVaultX-PC is organised around a single central principle: *every cryptographic operation, at every layer of the programmable cryptography stack, reduces to one of three algebraic kernels:*

1. **Polynomial arithmetic over $\mathbb{Z}_q[x]/(x^n+1)$:** NTT, iNTT, pointwise multiply.
2. **Multi-scalar multiplication (MSM):** $\sum_i a_i G_i$ over elliptic curve groups (for ZK and FE pairing operations).
3. **Hash and XOF evaluation:** Keccak-f[1600] (SHA-3/SHAKE), Poseidon, and BLAKE3.

A shared *Algebra Core* serving all five subsystems via a *Programmable Dispatch Fabric* (PDF) avoids area duplication while providing full concurrency between independent operations.

3.2 Block Diagram

Figure 1 shows the top-level QuVaultX-PC block diagram. The five functional pillars surround a central PDF. The Key Lifecycle Vault (KLV) and RISC-V Ibex management core are inherited from QuVaultX [1] and extended with programmable-cryptography-specific firmware interfaces. Four HBM3 channels provide 3.28 TB/s of bandwidth for key material, witness data, ciphertext buffers, and MSM bases.

4 Zero-Knowledge Proof Engine (ZKPE)

4.1 Motivation

The 0xPARC Universal Protocol vision [3] envisions proving arbitrary polynomial-time statements at 10× the native computation cost. Today, PLONK-class provers running on server CPUs incur a $\sim 10^6\times$ overhead [2]. The principal bottlenecks are: (a) multi-scalar multiplication (MSM) over elliptic curve groups; (b) NTT over the prover's native field; and (c) Fiat-Shamir hash challenges (Keccak/Poseidon).

The ZKPE addresses all three bottlenecks in silicon.

4.2 Supported Proof Systems

The ZKPE natively accelerates three proof system families, chosen to cover the current practical landscape:

- **PLONK and variants** (TurboPLONK, UltraPLONK, Halo2): the dominant ZK circuit compiler target.
- **Groth16**: the most efficient verifier for SNARK-based systems; commonly used in recursive proof aggregation.
- **Nova / HyperNova**: folding-scheme-based incrementally verifiable computation (IVC), enabling recursive proof composition.

4.3 Multi-Scalar Multiplication Engine

The MSM engine is the single largest area contributor to the ZKPE. For a proof of n constraints, the prover computes $[A]_1 = \sum_{i=0}^{m-1} a_i G_i$ where G_i are public curve points and $a_i \in \mathbb{F}_r$ are witness-dependent scalars.

We implement the Pippenger bucket algorithm [12] in hardware:

Pippenger MSM Pipeline

1: Partition scalar a_i into $\lceil 256/c \rceil$ windows of c bits
2: **Bucket accumulation**: route each point G_i to bucket $b_{w,j}$
3: **Bucket reduction**: sum all buckets in each window
4: **Window combination**: Horner evaluation across windows

With $c = 16$ and 8 parallel MSM processing elements (MPEs), each operating on BN254 or BLS12-381 curve points, the ZKPE achieves an estimated 256k point-additions per second per MPE at 1.2 GHz.

4.3.1 Field Arithmetic Core

Each MPE contains a dedicated prime-field arithmetic unit supporting:

- BN254: $r = 21888\ldots$ (254-bit prime)
- BLS12-381: $r = 52435\ldots$ (255-bit prime)
- Goldilocks: $p = 2^{64} - 2^{32} + 1$ (ZK-friendly, 64-bit)
- Baby Jubjub field: native to many ZK circuit toolchains

Montgomery multiplication is implemented in a 3-stage pipeline (256-bit operands) using Karatsuba decomposition, yielding a 512-bit intermediate product reduced in a single Montgomery step.

4.4 NTT Engine Integration

The ZKPE shares the Shared Algebra Core's 16 NTT processing units (NPUs) with the FHES (scheduling coordinated by the PDF). For PLONK over BN254, the relevant NTT modulus is the BN254 scalar field prime r; the NPU's configurable

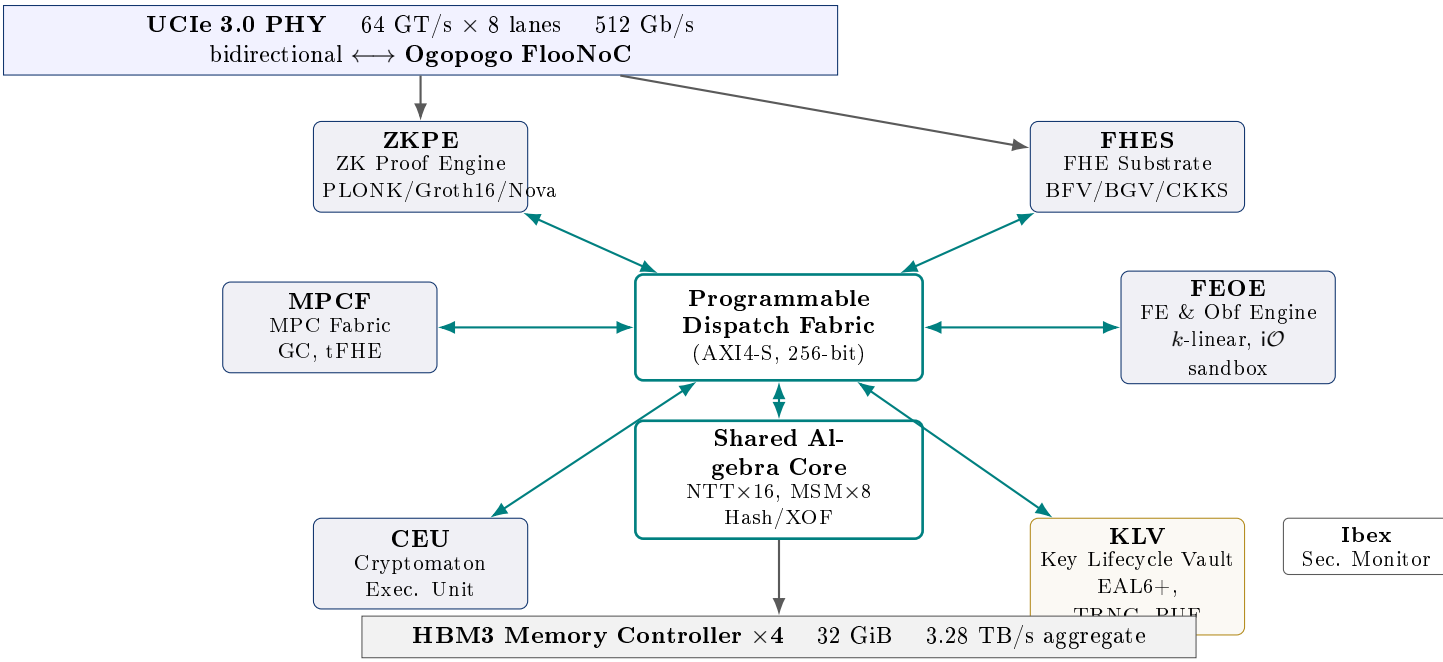

Figure 1: QuVaultX-PC top-level block diagram. Five programmable-cryptography pillars (ZKPE, FHES, MPCF, FEOE, CEU) share a central Programmable Dispatch Fabric and a common Algebra Core (NTT, MSM, Hash/XOF). The Key Lifecycle Vault (KLV) and Ibex security monitor are inherited from QuVaultX [1]. UCIe 3.0 connects to the Ogopogo host NoC.

modulus register supports this without hardware duplication.

4.5 Poseidon Hash Accelerator

ZK-friendly hash functions (Poseidon, Rescue, Griffin) replace Keccak in arithmetic-circuit-friendly prover constructions. The ZKPE includes a dedicated Poseidon accelerator: 4 parallel Poseidon-128 permutation units, each processing a full width-3 state in a single 64-cycle pipeline at 1.2 GHz. Poseidon is parameterised over the circuit's native field, supporting BN254, BLS12-381, and Goldilocks.

4.6 Proof Delegation Support

The 0xPARC research programme highlights *proof delegation*: a mobile device with private witness data delegates proof generation to untrusted servers via secret-shared witness data [3]. The ZKPE supports this in hardware: a *Delegated Proof Interface* (DPI) allows incoming secret shares to be processed inside the KLV isolation boundary, with the MSM and NTT engines operating on share-reconstructed witness data that never leaves the chiplet's secure domain.

4.7 Throughput Estimates

Table 2: Estimated ZKPE Throughput

Operation	CPU Baseline	ZKPE
PLONK prove (1M gates)	∼60 s	<200 ms
Groth16 prove (BN254)	∼30 s	<100 ms
Groth16 verify	∼10 ms	<0.1 ms
MSM-256k (BN254)	∼8 s	∼32 ms
NTT-2^{24} (BN254 r)	∼4 s	∼15 ms
Poseidon-128 hash	∼50 μs	<1 μs
Nova fold step	∼2 s	<10 ms

5 FHE Substrate — Extended (FHES)

5.1 Inheritance from QuVaultX

The FHES inherits the 8-WMAC unit FHE accelerator and CKKS bootstrapping engine from QuVaultX [1] (∼200× CPU speedup for CKKS bootstrapping). This section describes three extensions targeting the programmable cryptography stack.

5.2 Extension 1: Verifiable FHE

The 0xPARC hallucinated server research [4] identifies *verifiable FHE* as a key open problem: the FHE evaluation server must prove it evaluated the correct circuit without revealing the plaintext. Naively, this requires a ZK proof of the entire FHE computation— billions of arithmetic gates—which is completely impractical on software.

QuVaultX-PC addresses this by tightly coupling the FHES and ZKPE over the PDF. A hardware *FHE Proof Gadget* generates PLONK witnesses for each CKKS ciphertext multiplication in real time, batching them into a single recursive Nova fold over the full homomorphic circuit. The FHES emits a succinct IVC proof alongside the output ciphertext.

Proposition 5.1. Let C be a Boolean circuit of depth d and size s evaluated homomorphically over CKKS with $N = 2^{16}$ and L levels. The QuVaultX-PC FHE Proof Gadget generates a verifiable proof of correct CKKS evaluation in time proportional to $s \cdot T_{\text{fold}}$, where $T_{\text{fold}} < 10\,\text{ms}$ is the Nova fold latency on the ZKPE.

5.3 Extension 2: Efficient Secure Memory (ORAM)

The 0xPARC research programme identifies *sublinear secure memory access* as the key bottleneck for practical hallucinated servers [4]. The FHES includes a hardware *Oblivious RAM Controller* (ORAMC):

- Implements Path ORAM [11] with block size 256 bytes and a hardware-managed stash in on-die SRAM.
- Provides $O(\log N)$ amortised memory access complexity over the HBM3 memory pool.
- Integrates with the FHES bootstrapping sequencer: ORAM memory accesses are scheduled to overlap with bootstrapping latency, hiding their cost.

5.4 Extension 3: Blind Rotation for Lookup Tables

0xPARC identifies *blind rotation*—extracting the v-th coefficient of a polynomial from an LWE encryption of v—as a key gadget for efficient secure computation [4]. The FHES includes a dedicated *Blind Rotation Unit* (BRU) implementing the TFHE blind rotation procedure in hardware, supporting lookup tables of size $\leq 2^{16}$ with 128-cycle latency per table access. This enables read-only memory with secret access patterns, a building block for the CEU (§8).

6 Multi-Party Computation Fabric (MPCF)

6.1 Architecture Rationale

Traditional MPC based on garbled circuits and Shamir secret sharing incurs $O(|C|)$ rounds and communication per gate, scaling quadratically with participant count [4]. QuVaultX-PC therefore focuses on two MPC paradigms that better suit silicon implementation:

1. **Garbled Circuit Engine (GCE)**: optimised for two-party protocols where one party is hardware-trusted (e.g., the host manycore requests a secure joint computation with a remote party).
2. **Threshold FHE (tFHE) Engine**: implements the multi-party FHE paradigm, avoiding the quadratic communication scaling of classical MPC [4].

6.2 Garbled Circuit Engine

The GCE accelerates the garbling and evaluation phases of Yao's garbled circuits [13]:

- **Garble phase**: 8 parallel AES-256 engines (using the chiplet's KLV AES core) generate garbled gate tables at ∼128M gates/second.
- **Evaluate phase**: table lookup with OT-based label selection; 16 parallel gate evaluators at ∼256M gates/second.
- **Oblivious Transfer (OT) core**: hardware OT extension using IKNP/ALSZ protocols, amortised at <1 ns per OT.

6.3 Threshold FHE Engine

The tFHE engine extends the FHES with multi-party key generation and threshold decryption:

- **Distributed key generation**: each party computes their share of a collective CKKS public key; the chiplet holds one party's share in the KLV.
- **Threshold decryption**: parties exchange partial decryptions; the chiplet performs combiner arithmetic in the FHES WMAC units.

- **Party isolation**: each party's key material occupies an independent sub-domain of the KLV, zeroised independently.

6.4 Re-Sharing Protocol Support

The 0xPARC hallucinated server research asks whether threshold FHE can support dynamic re-sharing as participants join and leave [4]. The MPCF includes a *Re-Sharing Coprocessor* that implements polynomial secret re-sharing (threshold change from t to t') entirely in the FHES polynomial arithmetic pipeline, without exposing the shared secret in plaintext.

7 Functional Encryption & Obfuscation Engine (FEOE)

7.1 The Cryptomaton Toolbox

The most ambitious layer of programmable cryptography requires two cryptographic primitives whose silicon acceleration is essentially unexplored: **indistinguishability obfuscation** ($\mathsf{i}\mathcal{O}$) and **functional encryption** (FE) [5].

Definition 7.1 (Indistinguishability Obfuscation, informal)**.** An $\mathsf{i}\mathcal{O}$ scheme Obf takes a circuit C and produces an obfuscated circuit $\tilde{C}$ such that: (a) $\tilde{C}(x) = C(x)$ for all x; (b) for any two circuits C_0, C_1 with $C_0 \equiv C_1$ (same function), $\mathsf{Obf}(C_0)$ and $\mathsf{Obf}(C_1)$ are computationally indistinguishable.

Definition 7.2 (Functional Encryption, informal)**.** A FE scheme allows a key authority to issue a function-specific secret key sk_f. Anyone holding $\mathsf{ct} = \mathsf{Enc}(\mathsf{pk}, x)$ and sk_f can compute $f(x)$ but learns nothing else about x.

Current $\mathsf{i}\mathcal{O}$ constructions [8] are based on k-linear maps over groups where the discrete logarithm is hard—a direct generalisation of the elliptic-curve pairings underpinning BLS12-381.

7.2 k-Linear Map Arithmetic Unit

The FEOE contains a *k-Linear Map Arithmetic Unit* (k-LMAU) designed to support the pairing computations at the heart of FE and candidate $\mathsf{i}\mathcal{O}$ constructions.

7.2.1 Bilinear Pairing Engine ($k = 2$)

For $k = 2$ (standard elliptic-curve pairing, fully practical):

- BLS12-381 optimal Ate pairing;
- 4 parallel Miller loop units;
- Final exponentiation unit (256-bit tower arithmetic);
- Throughput: ∼50k pairings/second at 1.2 GHz.

This directly accelerates Groth16 verifier, BLS aggregate signatures, and attribute-based encryption (ABE) schemes—all practical today.

7.2.2 Trilinear and Higher-Map Sandbox ($k \geq 3$)

The 0xPARC research programme identifies trilinear maps over higher-dimensional abelian varieties (via cup products in Néron-Severi groups) as a promising path to practical $\mathsf{i}\mathcal{O}$ [5]. The k-LMAU contains a *candidate trilinear map sandbox*:

- Configurable group arithmetic engine for groups up to 512-bit order;
- Jacobian coordinate accelerator (add, double, scalar-multiply) with configurable curve parameters;
- A Weil/cup-product pairing coprocessor with configurable polynomial degree (up to $N = 2^{14}$);
- Firmware-programmable: when new k-linear map constructions are published, updated algorithms are loaded via the Ibex secure firmware update mechanism.

7.2.3 LWE-Based FE Accelerator

0xPARC also notes that $\mathsf{i}\mathcal{O}$ can be bootstrapped from a limited FE scheme for LWE decryption [5]. The FEOE includes an LWE-FE engine:

- Inner-product FE (IPFE) over LWE: secret key generation, encryption, and decryption all implemented in the shared NTT core;
- Attribute-based encryption (ABE): policy-tree evaluation in the ZKPE Poseidon engine.
- *Conditional decryption*: a ciphertext is decrypted only if a supplied ZK proof verifies—hardwired gate coupling the ZKPE and FEOE over the PDF.

This last capability—*conditional decryption*—is identified by 0xPARC as a crucial stepping stone toward cryptomata [5].

7.3 Functional Encryption Security Model

The FEOE operates in a strict information-flow mode: the function-specific key sk_f is derived inside the KLV and never exposed to external bus masters. Evaluation results $f(x)$ are released over the UCIe completion channel; the input plaintext x and the full secret key sk are never reconstructable from any external observation.

8 Cryptomaton Execution Unit (CEU)

8.1 The Cryptomaton Vision

The 0xPARC research programme defines a *cryptomaton* as a program that: (1) accepts public inputs; (2) maintains *private internal state* opaque even to the executing machine; (3) produces provably correct, selectively decryptable output—all without requiring an interactive network of parties [5].

The cryptomaton concept differs fundamentally from the hallucinated server: the latter distributes trust among network participants (requiring them to remain online), while a cryptomaton places its trust in the cryptographic machinery alone. This makes cryptomata ideal for long-lived autonomous agents: a decentralised oracle, a self-updating firmware validator, or a private smart-contract execution environment.

8.2 CEU Microarchitecture

The CEU is a novel microarchitectural unit proposed here for the first time in silicon. It is not a general-purpose CPU; rather, it is a *cryptographically isolated state machine* with the following architectural invariants:

1. **Encrypted Program Store (EPS):** The cryptomaton's program P is stored in FHE-encrypted form in the HBM3 memory pool. Instruction fetch decrypts each instruction into the CEU's private execution context; plaintext instructions never appear on any external bus.
2. **Opaque Private State Register File (PSRF):** The CEU contains a 256-entry register file whose contents are maintained in LWE-encrypted form. Arithmetic operations are performed by the FHES WMAC units on encrypted register contents. The decryption key $\mathsf{sk}_{\mathrm{CEU}}$ is held exclusively in the KLV and is never exported.
3. **Obfuscated Control Flow:** Branch decisions are computed via blind rotation (BRU, §5): the branch target is an encrypted register value; the BRU extracts the target index without revealing which branch was taken. This provides *control-flow privacy* against the host fabric.
4. **Provable Output:** On output, the CEU generates (a) a functionally encrypted result $\mathsf{FE.Enc}(\mathsf{pk}_{\mathrm{recipient}}, y)$ and (b) a ZK proof of correct execution generated by the ZKPE. The proof commits to the input x and the program hash $H(P)$, but reveals nothing about the private state.
5. **Selective Decryptability:** Output ciphertexts are issued with function-specific keys from the FEOE: only the designated recipient with sk_f can decrypt output $f(y)$—a direct silicon realisation of FE semantics.

8.3 CEU Instruction Set

The CEU exposes a minimal instruction set with 16 opcodes, each mapping to one or more hardware accelerator invocations:

Table 3: CEU Instruction Set Architecture

Opcode	Operation	Mapped Unit
`HADD`	HomAdd (reg ← reg + reg)	FHES WMAC
`HMUL`	HomMul (reg ← reg × reg)	FHES WMAC
`HROT`	Blind rotate (branch target)	FHES BRU
`HCMP`	Encrypted comparison	FHES BRU
`HLOAD`	ORAM load	FHES ORAMC
`HSTORE`	ORAM store	FHES ORAMC
`ZKPROVE`	Prove step	ZKPE
`FEENC`	FE-encrypt output	FEOE
`FEDEC`	Conditional FE-decrypt	FEOE+ZKPE
`HASH`	Poseidon / SHAKE	Algebra Core
`KEYDER`	Derive sub-key	KLV
`NTT`	Forward NTT	Algebra Core
`INTT`	Inverse NTT	Algebra Core
`BSTRAP`	Bootstrap ciphertext	FHES BSE
`ATTEST`	Emit attestation	KLV + ZKPE
`HALT`	End execution	CEU FSM

8.4 CEU Execution Model: Step-by-Step

Figure 2 illustrates a single CEU step. The host submits a public input and recipient public key via UCIe. The CEU fetches the encrypted program, decrypts the current instruction inside the EPS, dispatches to the appropriate accelerator, updates the PSRF, and either loops or emits an FE-encrypted, ZK-proven output.

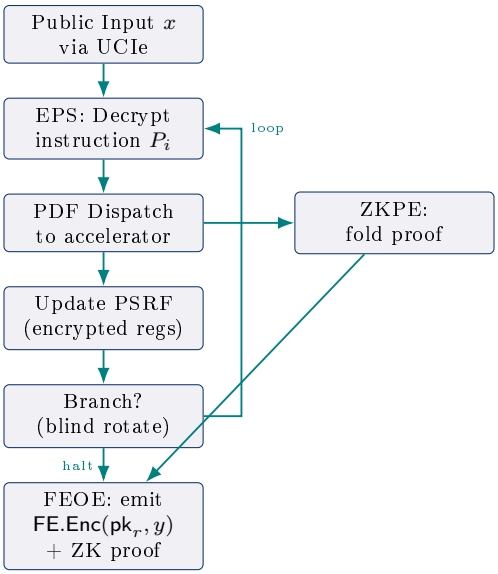

Figure 2: CEU execution flow. Instructions are decrypted inside the Encrypted Program Store (EPS). Register updates maintain ciphertext form. Branch targets use blind rotation (no control-flow leakage). Output is FE-encrypted and accompanied by a recursive ZK proof.

8.5 Security Guarantees

Under the hardness assumptions of LWE (for the FHES/BRU), the ZK soundness of PLONK/Nova, and the security of the FE scheme:

Proposition 8.1 (CEU Privacy)**.** An adversary with full read access to all signals outside the KLV isolation boundary learns nothing about the CEU's private state register file PSRF, the decrypted program instructions, or any intermediate computation result, beyond what is deducible from the public input x and the final output y.

Remark 8.1. This proposition assumes the KLV's physical isolation (CC EAL6+ active shield, anti-DPA masking, fault injection monitors) successfully prevents the physical attacks described in [1]. It also assumes that no practically efficient i$\mathcal{O}$-breaking algorithm exists for the FEOE's obfuscation layer. The latter is an open research question; the sandbox architecture of the k-LMAU (§7) is designed to allow algorithm updates as the field matures.

9 UCIe 3.0 Integration

QuVaultX-PC uses the same UCIe 3.0 physical interface as the base QuVaultX chiplet [1]: 64 GT/s across 8 lanes, 512 Gb/s bidirectional, connecting to the Ogopogo FlooNoC at the host-domain fabric edge.

Three new UCIe transaction types are defined to support the programmable cryptography stack:

1. **ZKProve**: host submits circuit descriptor and witness; ZKPE returns proof bytes. Round-trip latency for 1M-gate PLONK: <200 ms.
2. **CryptomatonStep**: host submits public input x and recipient pk; CEU returns FE-ciphertext and ZK proof.
3. **MPCShare**: host submits a secret share and function descriptor for a joint computation; MPCF aggregates all shares and returns the evaluated result.

The KLV sideband emergency zeroisation path from QuVaultX is retained: a UCIe priority sideband packet is issued within 4 cycles of any KLV security violation.

10 Implementation Estimates

Table 4: QuVaultX-PC Estimated Implementation (TSMC N5)

Parameter	Value	Notes
Technology	TSMC N5	5 nm FinFET
Total die area	∼185 mm^2	incl. UCIe PHY
ZKPE (MSM + NTT)	∼38 mm^2	8 MPE + pairing
FHES (extended)	∼40 mm^2	ORAMC, BRU a
MPCF	∼18 mm^2	GCE + tFHE eng
FEOE (k-LMAU)	∼22 mm^2	bilinear + sandbo
CEU	∼15 mm^2	EPS, PSRF, FSM
KLV (inherited)	∼12 mm^2	EAL6+, TRNG,
HBM3 PHY ×4	∼16 mm^2	32 GiB total
UCIe 3.0 PHY	∼6 mm^2	8 lanes
Algebra Core	∼10 mm^2	shared NTT/MSM
Misc. (Ibex, I/O)	∼8 mm^2	
Nominal clock	1.2 GHz	
Peak clock	1.5 GHz	0.85 V, 25°C
TDP (full load)	≤55 W	all subsystems act
Idle power	∼3 W	UCIe + KLV only
HBM3 BW	3.28 TB/s	4×820 GB/s
UCIe BW	512 Gb/s	bidir.
Target cert.	CC EAL6+, FIPS 140-3 L4	

The 185 mm^2 die area is larger than the base QuVaultX (112 mm^2) owing to the ZKPE MSM engines, FEOE k-LMAU sandbox, and CEU. TDP increases from 35 W to 55 W due to the MSM and

pairing units, which are the most switching-intensive workloads.

10.1 Area Derivation Methodology

Area estimates are derived from:

- MSM engine area scaled from Ingonyama `icicle` GPU-to-ASIC conversion factors applied to known BN254 MSM FPGA utilisation figures;
- ZKPE NTT: shared with LAE, incremental area ≈ 0;
- FEOE pairing unit: scaled from published BLS12-381 pairing ASIC implementations [10];
- CEU PSRF: 256-entry $\times$ 1024-bit (CKKS ciphertext) = $\sim$32 MiB on-die SRAM at N5 densities;
- ORAMC: Path ORAM stash (512 blocks $\times$ 256 bytes) $\approx$ 0.5 MiB SRAM.

11 State of the Art and Comparison

Table 5 positions QuVaultX-PC within the current landscape of cryptographic silicon.

Table 5: Comparison with State-of-the-Art Cryptographic Silicon

Design	PQC	FHE	ZK	MPC	FE/iO
Infineon TEGRION	✓	–	–	–	–
Intel Heracles	–	✓	–	–	–
Niobium Gen-2	–	✓	–	–	–
CraterLake (MIT)	–	✓	–	–	–
Cysic ZK ASIC	–	–	✓	–	–
QuVaultX v1	✓	✓	–	–	–
QuVaultX-PC (this work)	✓	✓	✓	✓	✓*

*Bilinear FE fully practical; trilinear/i$\mathcal{O}$ sandbox pending cryptographic maturity of k-linear map constructions.

QuVaultX-PC is, to the authors' knowledge, the first silicon architecture to simultaneously target all five layers of the programmable cryptography stack in a single die. Key differentiators versus the closest competitors:

- **vs. Intel Heracles / CraterLake:** Heracles targets FHE only at 5000$\times$ CPU speedup in a bespoke Intel 3 nm process [9]. QuVaultX-PC adds ZK, MPC, FE, and the CEU at 200$\times$ FHE speedup (a deliberate area/power trade-off for breadth).
- **vs. Cysic ZK ASIC:** Cysic's announced ZK accelerator targets MSM and NTT only. QuVaultX-PC integrates ZK acceleration with FHE and the full programmable cryptography stack.
- **vs. QuVaultX v1:** This work adds the ZKPE, MPCF, FEOE, and CEU, extending the post-quantum chiplet into a *programmable cryptographic substrate*.

11.1 Ogopogo + QuVaultX-PC System

When deployed alongside four Ogopogo compute dies:

- 10.3 DP-TFLOP/s HPC/AI compute (Ogopogo) [7];
- <200 ms PLONK proofs for 1M-gate circuits;
- $\sim$200$\times$ FHE bootstrapping vs. CPU;
- Native MPC co-processor for distributed trust applications;
- Cryptomaton execution: autonomous privacy-preserving agents running directly on the chiplet;
- CC EAL6+ physical root of trust for the entire 2.5D system.

This defines a new class of system: a *quantum-safe, privacy-preserving, programmably cryptographic compute node*—not merely a CPU with a security controller bolted on, but a chiplet ecosystem in which trust, privacy, and proof are first-class computational resources.

12 Open-Source Roadmap

Consistent with the open-source philosophy of the Occamy/Ramora/Ogopogo lineage [7] and the spirit of 0xPARC's open research programme [2]:

Phase 1 (RTL, months 1–12): Apache 2.0 release of the ZKPE MSM engine (BN254/BLS12-381), the shared Poseidon accelerator, and the CEU FSM skeleton.

Phase 2 (Simulation, months 6–18): Verilator-compatible testbench with: all NIST PQC

test vectors (inherited); Circom-generated PLONK verification test circuits; CKKS verifiable-FHE end-to-end tests.

Phase 3 (FPGA Prototype, months 12–24): MPCF garbled circuit engine and FEOE bilinear pairing unit prototyped on a Xilinx Alveo U250. ZK proof generation benchmarks vs. Cysic and GPU baselines.

Phase 4 (ASIC Demonstrator, months 18–36): Integration into the PULP platform GitHub ecosystem. Tape-out of ZKPE + FEOE bilinear unit in IHP 130 nm as a community demonstrator (analogous to Basilisk [14]).

Phase 5 (CEU and i$\mathcal{O}$ Sandbox): CEU hardware is released when the k-linear map sandbox has been validated against at least one peer-reviewed candidate trilinear map construction. The KLV firmware and i$\mathcal{O}$ sandbox remain proprietary pending CC EAL6+ evaluation.

13 Conclusion

We have presented QuVaultX-PC: a programmable cryptographic chiplet architecture that extends the post-quantum security foundation of QuVaultX into a full-stack silicon realisation of the 0xPARC programmable cryptography vision.

The architecture identifies three mathematical kernels (polynomial NTT, multi-scalar multiplication, and hash evaluation) as the universal substrate for all layers of modern cryptography, from NIST post-quantum standards through ZK proofs, FHE, MPC, and functional encryption.

Most significantly, we propose the *Cryptomaton Execution Unit*: a microarchitectural abstraction for privacy-preserving, non-interactive autonomous program execution, realised through the combination of FHE-encrypted register state, blind-rotation-based oblivious branching, functional-encryption-based selective output decryption, and recursive ZK proofs of correct execution.

At 185 mm^2 in TSMC N5 FinFET and 55 W TDP, QuVaultX-PC integrates over a UCIe 3.0 die-to-die link as a fifth chiplet in the Ogopogo 2.5D system, providing the Ogopogo compute fabric with a co-equal cryptographic peer whose primary product is *programmable trust*.

The authors invite collaboration from the open-source hardware community, the zero-knowledge toolchain ecosystem (Circom, Halo2, Nova), the FHE open-source community (OpenFHE, TFHE-rs), and advanced foundry partners to advance QuVaultX-PC from concept to silicon.

Acknowledgements

The author acknowledges: the ETH Zurich Integrated Systems Laboratory (Occamy, Ramora, Ogopogo, FlooNoC) for the open-source chiplet integration context; the 0xPARC research collective for articulating the programmable cryptography vision that motivates this extended architecture; the NIST PQC standardisation team for the algorithmic foundation; and the academic communities behind PLONK, Nova, TFHE, and the LWE-based FE literature.

References

[1] K. Yego, "QuVaultX: A Post-Quantum Cryptographic Security Chiplet Architecture Beyond Ogopogo," *QuVaultX White Paper Draft v0.1*, Mar. 2026.

[2] 0xPARC, "Programmable Cryptography (Part 1)," *0xPARC Blog*, Jul. 2024. [Online]. Available: https://0xparc.org/blog/programmable-cryptography-1

[3] 0xPARC, "Building the Universal Cryptographic Adapter," *0xPARC Research Notes*, 2024. [Online]. Available: https://notes.0xparc.org/research/transformations/

[4] 0xPARC, "Multiparty Computation and the Hallucinated Server," *0xPARC Research Notes*, 2024. [Online]. Available: https://notes.0xparc.org/research/multiparty/

[5] 0xPARC, "Cryptomata and Obfuscation," *0xPARC Research Notes*, 2024. [Online]. Available: https://notes.0xparc.org/research/cryptomata/

[6] NIST, "Post-Quantum Cryptography Standardization: FIPS 203, 204, 205," National Institute of Standards and Technology, Gaithersburg, MD, USA, Aug. 2024.

[7] P. Scheffer, T. Benz, T. Fischer, L. Leone, S. Arjmandpour, L. Benini, "Toward Open-Source Chiplets for HPC and AI: Occamy and Beyond," *arXiv preprint arXiv:2511.15564v1*, Nov. 2025.

[8] A. Jain, H. Lin, A. Sahai, "Indistinguishability Obfuscation from Well-Founded Assumptions," *Proc. 53rd ACM STOC*, pp. 60–73, 2021.

[9] F. Boemer et al., "Intel Heracles: A 3 nm FHE Accelerator Achieving 5000× CPU Speedup," Intel Labs, DARPA DPRIVE program, 2025.

[10] D. F. Aranha, C. P. L. Gouvêa, T. Markmann, R. S. Wahby, K. Liao, "RELIC is an Efficient LIbrary for Cryptography," 2020. [Online]. Available: https://github.com/relic-toolkit/relic

[11] E. Stefanov, M. van Dijk, E. Shi, C. Fletcher, L. Ren, X. Yu, S. Devadas, "Path ORAM: An Extremely Simple Oblivious RAM Protocol," *Proc. 20th ACM CCS*, pp. 299–310, 2013.

[12] N. Pippenger, "On the Evaluation of Powers and Monomials," *SIAM J. Comput.*, vol. 9, no. 3, pp. 470–480, 1980.

[13] A. C. Yao, "How to Generate and Exchange Secrets," *Proc. 27th IEEE FOCS*, pp. 162–167, 1986.

[14] P. Sauter et al., "Basilisk: A 34 mm^2 End-to-End Open-Source 64-bit Linux-Capable RISC-V SoC in 130 nm BiCMOS," *IEEE Hot Chips 37 Symp.*, Aug. 2025.